Codex-to-GitHub Academic Website Workflow

A tutorial for turning a Word CV into a static researcher website and publishing it through GitHub Pages

Jewoong Moon (The University of Alabama)

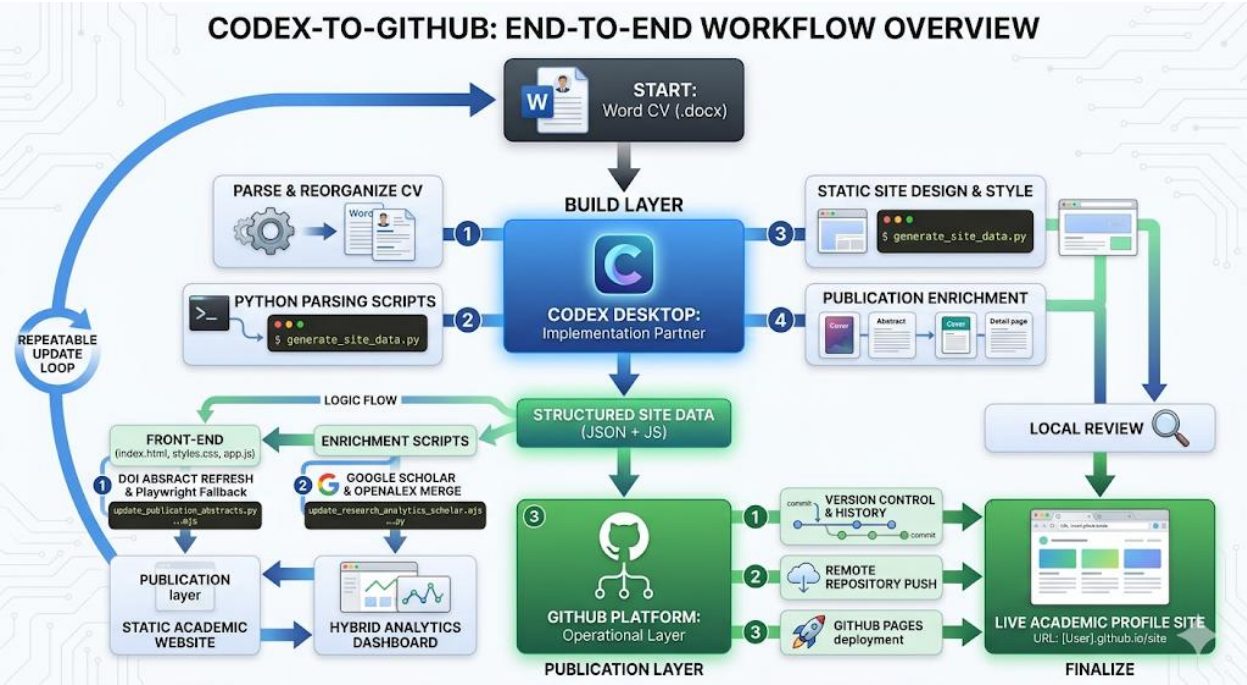
April 2026

1. Document Purpose

This tutorial explains the complete workflow used to build and maintain an academic website by working with Codex and GitHub together. The key idea is not only that a Word CV becomes the source of truth for the website, but also that Codex serves as the implementation partner for parsing, rendering, enrichment, analytics, refinement, and deployment tasks. GitHub then becomes the publishing and version-control layer that turns those changes into a live site through GitHub Pages.

The central question of this workflow is: how does a researcher actually work with Codex, step by step, and then move those results into GitHub in a disciplined way? This document answers that question by describing the architecture, the interaction pattern with Codex, the script stack, the GitHub publishing steps, and the repeatable update loop.

This tutorial is written primarily for researchers, graduate students, and academic project teams who are comfortable working with files and folders and who have at least light familiarity with Python concepts. A reader does not need to be an expert programmer to follow the logic of the workflow, but the implementation sections will be easier to reproduce if the reader is already somewhat comfortable with scripts, local project folders, and basic GitHub operations.



2. Starting Scenario: Beginning with a CV in Word

The most important assumption in this tutorial is that the beginner usually does not start with a parser, a JSON schema, or a GitHub Actions workflow. The beginner starts with a CV already written in Microsoft Word. In many real cases, that CV is the only organized source document the researcher has. It may already contain appointments, education, publications, grants, teaching, honors, affiliations, and service, but it still exists as a document meant for human reading rather than a data source meant for a website.

That means the most natural starting workflow is not "build automation first." The natural workflow is:

1. open the CV in Word
2. copy and paste the relevant CV content into Codex Desktop, or place the Word file into the project workspace
3. ask Codex to create the first website version from that material
4. review the first visible website structure
5. then ask Codex to formalize the CV as the source of truth by reading the `.docx` directly
6. move the project into a repeatable GitHub-based workflow

This distinction matters. For beginners, there are really two phases:

- an exploratory phase, where Word content is pasted or shared so Codex can generate the first working version
- a workflow phase, where the actual `.docx` becomes the stable input file for regeneration

If a tutorial skips that transition and jumps directly into parser architecture, the flow feels less natural than the way people actually begin. This document therefore treats "I already have my CV in Word" as the real starting point.

3. Core Workflow Idea

The original problem was straightforward. Updating a CV and a website separately became repetitive and inefficient. The solution was to treat the CV as the single source of truth, then use Codex to build and refine a script-driven static site that reads structured data generated from that CV. Once the project exists in a GitHub repository, the workflow becomes repeatable: update the CV, ask Codex to regenerate the site data and supporting assets, review locally, push to GitHub, and publish through GitHub Pages.

This means the workflow has three tightly connected layers:

1. Codex as the implementation and iteration partner
2. Python and Playwright scripts as the transformation pipeline
3. GitHub as the versioning, deployment, and publication layer

4. What Codex Does in This Workflow

Codex is most useful here not as a one-shot generator, but as a development collaborator. In practice, Codex was used to:

- inspect the project structure
- parse and reorganize CV content
- implement and refine Python parsing scripts
- design the static site structure
- style the front end
- add publication cards and detail pages
- attach journal cover images and service thumbnails
- build a hybrid analytics dashboard
- troubleshoot local rendering issues
- commit and push changes to GitHub

The important lesson is that the workflow is iterative. Instead of asking for the whole website in one prompt, the process works better when each layer is added or refined through concrete requests.

5. What GitHub Does in This Workflow

GitHub has several roles in this system.

- It stores the project files and script history.
- It preserves incremental revisions as commits.
- It acts as the remote origin for pushes from the local workspace.
- It provides GitHub Pages as the hosting layer for the final static site.
- It supports the future automation path through GitHub Actions.

In other words, Codex is the build partner, while GitHub is the durable operational layer.

---

6. Beginner Setup Before Starting

Before a beginner can reproduce this workflow, there are a few practical prerequisites that need to be understood clearly. These setup details matter because the workflow depends on both local tooling and GitHub-based publishing.

6.1 Start by Installing the Codex Desktop App

For a beginner, the cleanest starting path is the Codex desktop app rather than the CLI. OpenAI's current getting-started page reduces the first experience to three steps: open Codex, sign in with a ChatGPT account, and select a folder or Git repository on the computer where Codex will work. OpenAI's product and help documentation also now describe the Codex app as a first-class client alongside the CLI, IDE extensions, and Codex web. In practice, this makes the desktop app the easiest entry point for a tutorial because it avoids asking a new user to learn terminal setup before they even begin collaborating with Codex.

One small clarification is important for beginners here. Codex is not simply another name for ChatGPT, but it is tied to the user's ChatGPT account and plan for access. In practical terms, that means the user signs in to the Codex app with a ChatGPT account, but then uses Codex as a project-oriented coding environment rather than as a general-purpose chat interface.

That means the main beginner path in this document should be framed like this:

1. install the Codex desktop app
2. sign in with a ChatGPT account
3. open the local project folder or local Git repository
4. start prompting Codex inside that project

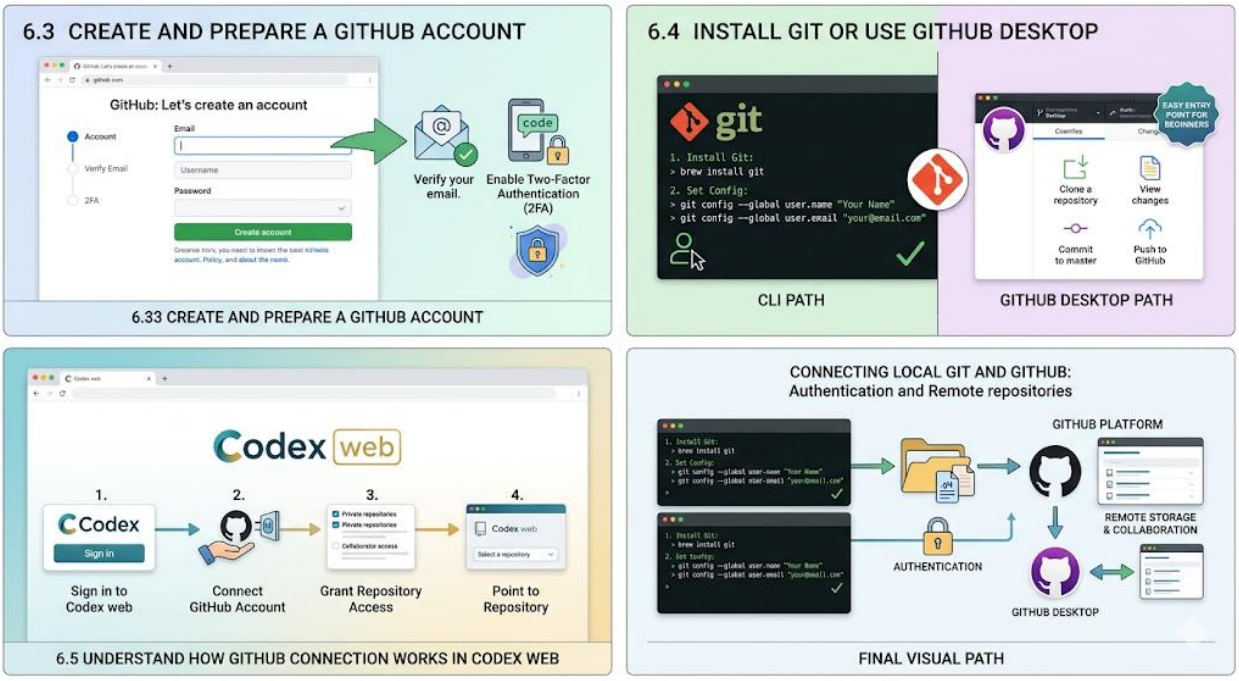
This path is especially appropriate for the kind of CV-to-website workflow described here because the project depends on a local Word CV file, local scripts, local previewing, and iterative file edits. The desktop app fits that local, project-centered workflow naturally.

6.2 Treat the CLI as a Secondary or Advanced Path

The Codex CLI still matters, but it should be presented as the secondary path rather than the first one for beginners. The official OpenAI CLI documentation states that the CLI can be installed with `npm i -g @openai/codex`, then started with `codex`, after which the user signs in with a ChatGPT account or an API key. This is useful for users who are already comfortable in a terminal and want a lightweight command-line workflow.

However, beginners should be told clearly that they do not need to start there. The desktop app already supports working on local folders and repositories, which is enough for most first-time users following this tutorial. The CLI becomes more useful later when a user wants terminal-native workflows, tighter scripting habits, or integration with other developer tools.

For Windows users, one additional point should be noted. OpenAI's current CLI documentation still notes that Windows support is experimental and recommends using Codex in a WSL2 workspace for the best Windows experience. That is another reason the desktop app should be the main recommendation in a beginner-facing guide, while the CLI is introduced as an optional next step.



6.3 Create and Prepare a GitHub Account

GitHub also has beginner requirements that should be stated clearly in the document. GitHub's official getting-started documentation says that a new user should create a GitHub account, verify the email address, and then continue with repository creation and collaboration features. GitHub also strongly recommends enabling two-factor authentication. For beginners, this matters because an unverified account or poorly configured account can block basic tasks such as creating repositories or authenticating from local Git.

6.4 Install Git or Use GitHub

Beginners also need to know that GitHub and Git are not the same thing. GitHub is the hosting and collaboration platform, while Git is the version control tool used locally on the computer. GitHub's official setup guide states that working locally with repositories requires downloading, installing, and configuring Git. That includes setting a Git username and email address and choosing how to authenticate to GitHub.

However, not every beginner should start with the command line. GitHub's own documentation points out that GitHub or Github Desktop is often easier for newcomers because it provides a graphical interface for cloning repositories, viewing changes, making commits, and pushing to GitHub. So for the document, it is worth explicitly saying that beginners have two valid paths:

- command line with Git
- GitHub for a more visual workflow

6.5 Install the Other Local Prerequisites

This workflow also depends on a few non-Git tools that a beginner may not think about immediately.

- Python 3 is needed for the CV parser and document-generation scripts.
- `python-docx` is needed for reading the Word CV and generating Word outputs.
- Node.js and npm are needed for the Codex CLI and Playwright-based scripts.
- Playwright is needed for the Google Scholar and abstract fallback layers.

Without these tools, the workflow may look conceptually clear but still fail at the implementation stage.

6.6 Know the Difference Between Local Work and Published Work

Another beginner issue is understanding the difference between local editing and public publishing. Codex may help modify the local repository, but the public site does not change until those edits are committed and pushed to GitHub and then published through GitHub Pages. That distinction needs to be made explicit because beginners often assume that a local preview and a public website are automatically the same thing.

6.7 Understand GitHub Pages as Static Hosting

The GitHub Pages layer should also be explained early for beginners. GitHub's official Pages documentation describes GitHub Pages as a static site hosting service that publishes HTML, CSS, and JavaScript files directly from a repository. That matters because this project is intentionally a static site. The website is not updated by a backend server after deployment. Instead, the repository contents are updated and GitHub Pages republishes the static output.

6.8 Understand the Recommended Final Automation Path

For beginners, it is also useful to clarify that the preferred long-term automation model is GitHub Actions rather than a separate always-running backend. GitHub's documentation explains that GitHub Actions can automate build, test, and deployment pipelines, and GitHub Pages can also be deployed through custom workflows. This makes GitHub Actions a natural final layer for a project like this, where a changed CV file should trigger parsing, enrichment, rebuild, and redeployment.

6.9 Other Things Beginners Should Know

There are several additional points that deserve mention in a beginner-oriented version of this tutorial.

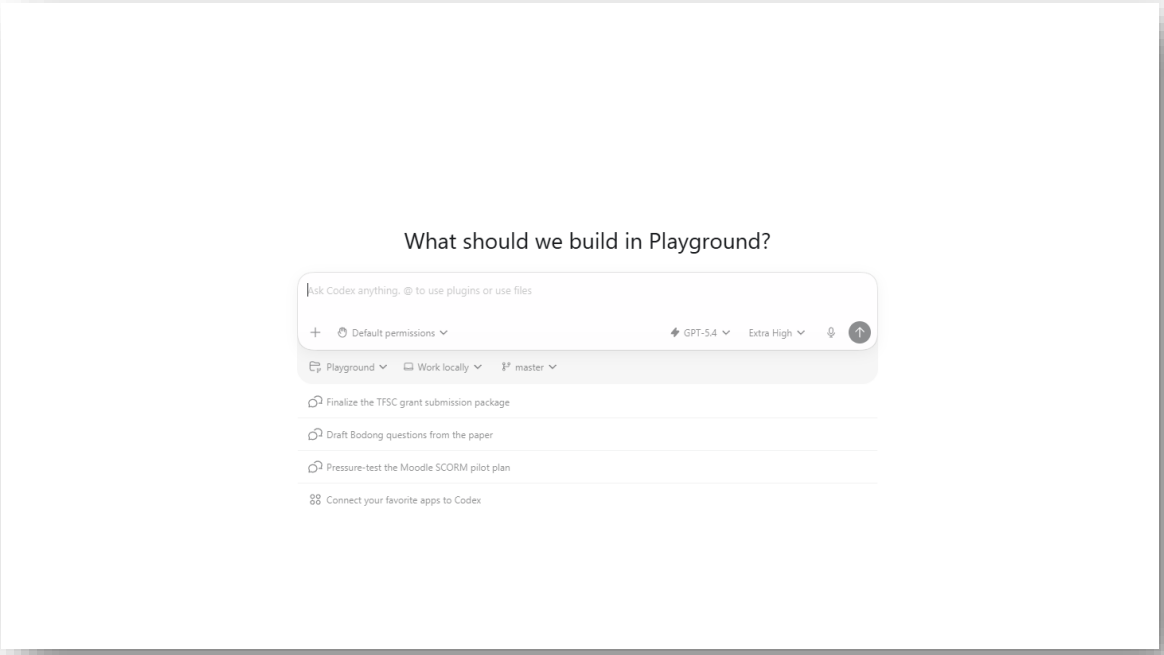
- A repository is the project container in GitHub, not just a folder name.
- Commits should be small and meaningful so changes are easier to review.
- Pushing to GitHub is not the same thing as publishing to GitHub Pages, though it may trigger publishing depending on the setup.
- GitHub Pages has both user/organization sites and project sites, and the site URL differs depending on which one is used.
- If the workflow later uses GitHub Actions, the repository's Actions settings must allow the workflow to run.
- Users should review Codex-generated changes before committing and pushing, especially in beginner stages.

Taken together, these setup points are important enough that they should appear near the beginning of the tutorial rather than being treated as advanced side notes.

6.10 A Beginner-Friendly Codex Desktop Installation Walkthrough

Because this document now treats the Codex desktop app as the main beginner path, it is worth writing the installation flow in a way that can later be paired with screenshots or live demonstration slides. A beginner should be able to follow the setup in this order:

1. go to the official Codex getting-started page or Codex app page
2. download and install the Codex desktop app for the operating system
3. launch the app
4. sign in with the ChatGPT account that has Codex access
5. choose or open a local folder or Git repository
6. start the first task inside that project



If this is later turned into a slide deck or workshop handout, the screenshots should probably correspond to those same steps. In other words, the installation sequence should not be explained abstractly. It should be shown as a visible path from "download the app" to "Codex is now looking at my actual project folder." For a beginner, that transition is what makes the tool feel usable.

An instructor-facing explanation can also clarify what each screen is doing. The download screen is where the user gets the correct app. The sign-in screen is where Codex is linked to the user's ChatGPT plan. The folder-selection screen is where Codex is given the actual working context. The first task screen is where the beginner learns that Codex is not just a chatbot window but a project-oriented coding environment.

6.12 The Simplest GitHub Workflow a Beginner Should Learn

For this tutorial, a beginner does not need to master every Git concept immediately. The smallest practical loop is enough:

1. make or review changes locally
2. check what changed
3. commit with a meaningful message
4. push to GitHub
5. pull before the next round if the remote changed

That means the tutorial should explicitly teach a minimum operational vocabulary:

- repository

- local clone
- commit
- push
- pull
- origin
- main branch

For beginners using GitHub Desktop, this can be explained as a visual cycle: edit files locally, review changed files in the app, write a commit summary, commit locally, then click push. For beginners using Git on the command line, the parallel commands can be introduced later, but the conceptual loop stays the same.

The most important beginner habit is to avoid treating GitHub as a giant "save" button with no review step. The workflow in this project depends on reviewing what Codex changed, deciding whether it is correct, then committing intentionally. That review-before-push behavior is one of the central disciplines that makes Codex and GitHub work well together.

6.13 How a GitHub Repository Fits into the Codex Desktop Workflow

Many beginners assume they need to start from Codex web or paste a GitHub URL directly into Codex before they can do anything meaningful. For this tutorial, that is not the easiest path. The simplest project flow is usually:

1. create or identify the GitHub repository
2. clone it locally with Git or GitHub Desktop
3. open the cloned local folder in Codex desktop
4. let Codex work on the local files
5. review, commit, and push the results back to GitHub

This matters because the project described in this tutorial is local-first. It includes a Word CV file, Python scripts, generated JSON, static front-end files, local previewing, and local document generation. All of those are easier to manage when Codex is pointed at the local working copy of the repository rather than only a cloud-connected view.

Once the user understands this local-first pattern, the role of GitHub becomes clearer. GitHub is where the durable project history lives and where GitHub Pages publishes the site. Codex desktop is where the actual iterative work happens. The connection between them is not mysterious: it is just the normal local-repo-to-remote-repo workflow.

6.14 A First Practical Exercise for Beginners

If this tutorial is used in a workshop, the first exercise should be intentionally small. A beginner should:

1. install and open Codex desktop

2. sign in
3. clone a GitHub repository locally or create a small local project folder
4. open the Word CV and copy one representative section or the full CV text
5. open the local folder in Codex desktop
6. paste the CV content into Codex and ask for a first academic website draft
7. review the generated files or visible change
8. commit and push the result to GitHub once it is understandable and reviewable

This kind of exercise is important because it teaches the actual interaction loop instead of only describing architecture. Before a beginner learns CV parsing, Playwright enrichment, or GitHub Pages deployment, they should first experience the basic pattern:

- open a project
- prompt Codex clearly
- inspect changes
- save and publish the result through GitHub

Once that loop feels normal, the more advanced CV-to-website automation workflow becomes much easier to understand.

6.15 Absolute Beginner Checklist

The following checklist is written for someone who may be new to both Codex and GitHub. It can be used as a one-page readiness check before starting the full CV-to-website workflow.

Account and access checklist

- [ ] I have a ChatGPT account with access to Codex.
- [ ] I have a GitHub account.
- [ ] My GitHub email is verified.
- [ ] I have enabled two-factor authentication on GitHub or at least understand that GitHub strongly recommends it.

Local software checklist

- [ ] I installed the Codex desktop app.
- [ ] I can open Codex desktop and sign in successfully.
- [ ] I installed Git or GitHub Desktop.
- [ ] I installed Python 3.
- [ ] I installed Node.js and npm.
- [ ] I understand that Playwright may need to be installed later for metadata and analytics workflows.

Project setup checklist

- [ ] I created a project folder or cloned a GitHub repository locally.
- [ ] I can find that folder on my own computer.
- [ ] I know where my Word CV file is stored.
- [ ] I opened that local folder or repository in Codex desktop.
- [ ] I understand that Codex works on local project files, not only on pasted text.

First Codex interaction checklist

- [ ] I know that I can begin by pasting CV text from Word into Codex Desktop.
- [ ] I asked Codex to inspect the project and explain the file structure.
- [ ] I asked Codex to make one small, visible change.
- [ ] I reviewed the changed files instead of accepting them blindly.
- [ ] I understand which files were changed and why.

Git and GitHub checklist

- [ ] I know the difference between a repository, a commit, a push, and a pull.
- [ ] I can review changed files in GitHub Desktop or Git before committing.
- [ ] I can write a short commit message that describes the change.
- [ ] I can push local commits to GitHub.
- [ ] I know that pushing to GitHub is not always the same thing as publishing a website.

Publishing checklist

- [ ] I understand that GitHub Pages hosts static files from a repository.
- [ ] I know where the published site URL comes from.
- [ ] I can tell the difference between a local preview and the public website.
- [ ] I know that future automation will most likely be handled through GitHub Actions.

Workflow mindset checklist

- [ ] I understand that the CV is the source of truth in this project.
- [ ] I understand the difference between the early paste-from-Word stage and the later `.docx` automation stage.
- [ ] I understand that Codex is the implementation partner, not the only source of judgment.
- [ ] I understand that I should review changes before committing and pushing.
- [ ] I am ready to follow the repeatable loop: update source -> ask Codex to modify or regenerate -> review locally -> commit -> push -> publish.

For a workshop setting, this checklist can be used at the start of the session, midway through the first practical exercise, and again before participants attempt the full CV-to-GitHub workflow on their own projects.

---

7. End-to-End Architecture

At a high level, the workflow looks like this:

1. Start with a Word CV in Word.
2. Use Codex Desktop to turn that CV content into a first working website draft, usually by pasting or sharing the CV material.
3. Once the shape of the site is correct, place the actual `.docx` into the project and treat it as the stable source file.
4. Use Python to parse the CV into structured site data.
5. Use the static front end to render that site data.
6. Use publication enrichment scripts to add abstracts and journal visuals.
7. Use Google Scholar and OpenAlex scripts to build a research analytics dashboard.
8. Review the site locally.
9. Commit and push the updated project to GitHub.
10. Publish or refresh the site through GitHub Pages.

The live site remains static, but the update pipeline is script-driven and iterative.

7.1 Constraints to Keep in Mind Early

Before going deeper into the file structure and automation layers, it helps to acknowledge the main limitations up front.

- `.docx` parsing depends on section consistency in the CV.
- Abstract extraction remains uneven across publishers.
- Google Scholar does not offer a simple official API for this use case.
- Some analytics fields require approximation or hybrid matching.
- Future automation through GitHub Actions still needs to be formalized.

These constraints are manageable, but they shape expectations for anyone trying to reproduce the workflow.

8. Project Inputs and Outputs

8.1 Real Beginner Inputs

At the very beginning, a beginner usually has these things rather than a finished automation pipeline:

- a CV in Microsoft Word
- CV text that can be copied and pasted into Codex Desktop
- a local project folder or a cloned GitHub repository
- optional supporting files such as a headshot, journal covers, or logos

The practical importance of this list is that the workflow can begin before the parser is fully built. A beginner can first ask Codex to make a visible working version from pasted CV content, then formalize the system afterward.

8.2 Stable Automation Input

- `assets/current-cv.docx`

Once the workflow is stabilized, this becomes the preferred parser input path used by the orchestration scripts.

8.3 Generated Files

- `assets/site-data.generated.json`
- `assets/site-data.js`
- `assets/publication-abstracts.json`
- `assets/research-analytics-scholar.json`
- `assets/research-analytics.json`

8.4 Main Front-End Files

- `index.html`
- `styles.css`
- `app.js`
- `analytics.js`
- `publication.html`
- `publication.js`

8.5 Main Script Files

- `scripts/generate_site_data.py`
- `scripts/update_publication_abstracts.py`
- `scripts/update_publication_abstracts_playwright.mjs`
- `scripts/update_research_analytics_scholar.mjs`
- `scripts/update_research_analytics.py`
- `scripts/update_site_from_cv.py`

---

9. The Practical Codex Working Loop

The most important part of this tutorial is the interaction pattern with Codex. The workflow did not begin with code alone. It began with files, goals, and a sequence of increasingly concrete instructions.

9.1 Step A: Start with What Already Exists in Word

For the scenario assumed in this document, the first real asset is the CV already written in Word. A beginner does not need to wait until the parser exists. The natural first step is to open the Word CV, identify the most important sections, and bring that content into Codex in one of two ways:

- paste the relevant CV text directly into Codex Desktop
- place the Word file itself in the workspace and tell Codex where it is

For an early prototype, the paste-based route is often the easiest because it reduces setup friction. For the long-term workflow, the file-based route is better because it supports regeneration.

9.2 Step B: Give Codex the Core Inputs

Once the user is in Codex Desktop, the next practical step is to make the key materials available in the workspace and reference them clearly. In this project, those included:

- the Word CV
- a headshot
- journal cover images
- the target repository folder

Once those files are available, Codex can operate directly in the workspace instead of generating abstract examples.

9.3 Step C: Ask Codex for the First Functional Version

The first Codex request should not try to solve everything. A better opening request is something like:

"Build a GitHub Pages-style academic CV website from my attached CV."

That gets the project moving by creating the first static site skeleton. At this stage, the goal is not perfection. The goal is to establish:

- a site structure
- a front-end layout
- a basic content model
- a live preview

For beginners working from Word, this stage is where pasted CV content or the shared `.docx` becomes a visible website draft. That visible result is important because it gives the user something concrete to refine before worrying about automation.

9.4 Step D: Shift from Pasted Content to a Repeatable Workflow

After the initial site exists, the next major Codex instruction is to stop treating the content as hand-authored front-end text or one-time pasted text and start treating the CV as the data source. That is the point where Codex is asked to implement the parser and generation pipeline. This is where the project changes from "a CV website prototype" into "a CV-driven website workflow."

Typical request pattern:

"Make it so that if I update the CV, the website can regenerate from that file rather than requiring manual edits."

This is also the point where the tutorial should explain the transition clearly: pasted Word content is a practical way to begin, but the `.docx` file should become the long-term source of truth.

9.5 Step E: Iterate in Visible Layers

The project improves fastest when Codex is given small, visible, testable requests. In this workflow, that included:

- update corresponding author labels
- add journal cover thumbnails
- widen the layout
- make sections tab-based instead of long-scroll
- add publication abstract previews
- move full abstracts to detail pages
- add service thumbnails
- make affiliations and appointments collapsible
- build analytics as a separate tab

Each of those requests changed a visible part of the site and was easy to verify.

9.6 Step F: Use Codex for Both Implementation and Cleanup

Codex was also useful for:

- fixing layout bugs
- resolving repeated publication matches
- improving metadata alignment
- hiding stale loading placeholders
- clarifying source labels in the analytics dashboard

This is an important part of the workflow. Codex is not only for "create" actions; it is equally useful for "refine," "correct," and "stabilize" actions.

---

10. Step-by-Step Technical Workflow

10.1 Stabilize the Word CV as the Source File

Once the first website version is visible, the workflow should stop depending on pasted text alone. The next step is to place the Word CV into the project in a stable location such as `assets/current-cv.docx`. That makes the project reproducible. At that point, the user is no longer relying on memory or manual copy-paste for every update. The document becomes the durable source file for the site.

10.2 Parse the Word CV into Structured Data

The parsing layer is implemented in Python using `python-docx`. The script `scripts/generate_site_data.py` reads the `.docx` file and extracts major academic sections such as appointments, education, publications, grants, teaching, affiliations, service, and honors. The result is saved into JSON and browser-ready JavaScript.

This is the first point where the CV becomes machine-readable site content rather than a static document.

10.3 Render the Static Site

The front end is a static HTML, CSS, and JavaScript site. The site uses tab-based navigation for Overview, News, Publications, Analytics, Grants, Teaching, and Service. A left profile rail contains the headshot, expertise, affiliations, appointments, and honors. The main content panels read the generated site data.

Codex was used to progressively redesign the structure so that it felt like a formal academic profile site rather than a portfolio or single-page resume.

10.4 Build the Publication Layer

The publication layer was expanded beyond a bibliography. Codex was used to add:

- publication cards
- pagination
- journal front-cover thumbnails
- abstract previews
- publication detail pages
- corresponding author indicators

This stage is one of the best examples of Codex's value in academic web work: it can translate a dense publication list into a richer, more navigable presentation.

10.5 Refresh Publication Abstracts

The project uses two stages for abstract enrichment.

1. `scripts/update_publication_abstracts.py` attempts DOI-driven collection.
2. `scripts/update_publication_abstracts_playwright.mjs` provides a Playwright fallback for difficult publisher pages.

This design was added because some publishers expose abstract information more reliably than others. Codex was used to implement both the DOI-first pass and the browser-based fallback.

10.6 Build the Analytics Layer

The analytics dashboard uses a hybrid model.

Google Scholar, scraped through `scripts/update_research_analytics_scholar.mjs`, provides:

- total citations
- h-index
- i10-index
- annual citation trend
- per-paper cited-by counts

OpenAlex, merged through `scripts/update_research_analytics.py`, provides:

- venue distribution
- topic clusters
- coauthor network
- open-access indicators

Codex was used to add the D3-based visualizations, source labeling, and snapshot wording. This turned the site from a CV mirror into a researcher dashboard.

10.7 Orchestrate the Full Update

The orchestration script `scripts/update_site_from_cv.py` ties the pipeline together. This is where the workflow becomes especially reusable. Instead of manually running multiple unrelated commands, the user can update the CV and run one script-driven flow.

The intended sequence is:

1. sync the latest CV
2. generate site data
3. refresh Google Scholar analytics
4. merge OpenAlex analytics

5. refresh publication abstracts
6. optionally run Playwright fallback

This is one of the most important pieces of the project because it turns a collection of scripts into an actual workflow.

---

11. How GitHub Fits into the Day-to-Day Workflow

GitHub enters the workflow once the local project is in a useful state. At that point, Codex is used not only to edit files, but also to help with repository tasks.

The practical GitHub loop looks like this:

1. review local changes
2. ask Codex to stage the relevant files
3. commit with a clear message
4. push to the remote repository
5. verify GitHub Pages deployment

This means the tutorial is not only about coding. It is also about disciplined project progression.

11.1 Example GitHub-Oriented Codex Requests

- "Push the latest changes to GitHub."
- "Create a clean commit for the analytics dashboard update."
- "Enable or verify GitHub Pages deployment."
- "Check whether the latest site has been published."

In practice, these prompts are effective because Codex can inspect the repo, understand changed files, and complete the push-and-deploy sequence without forcing the user to handle every low-level step manually.

12. GitHub Pages as the Publication Layer

The live site is published through GitHub Pages. That keeps the hosting model simple because the site itself is static. Once the project is pushed to the repository, GitHub Pages serves the generated front-end files directly.

This model works well for academic profiles because:

- the hosting is straightforward
- the site is easy to version
- the published URL is stable
- the build logic remains in the repository

The user does not need to maintain a separate runtime backend.

13. Why GitHub Actions Is the Preferred Final Automation Layer

Even though the current workflow is already script-driven, the final logical step is GitHub Actions. That would make the update cycle even cleaner:

1. update one CV file in the repository
2. let GitHub Actions run the parser and refresh scripts
3. rebuild the generated files
4. let GitHub Pages redeploy the site

This preserves the static-site model while removing even more manual steps. For a researcher who updates publications, grants, and service activities regularly, GitHub Actions is the most natural long-term automation layer because it lives in the same repository and fits the GitHub Pages publishing model.

14. Typical Prompt Progression for Codex

One reason this workflow worked well is that the prompts became more specific over time. A realistic progression looks like this:

14.1 Initial Build Prompt

"I am pasting my CV from Word below. Create the first GitHub Pages-style academic website version from this CV content."

14.2 Data-Driven Upgrade Prompt

"Now stop treating the pasted CV text as one-off content. Use the Word CV as the source of truth and generate structured site data from it."

14.3 Publication Enrichment Prompt

"Turn the publication section into cards with journal covers, abstract previews, and detail pages."

14.4 Analytics Prompt

"Add a researcher analytics dashboard using Google Scholar for citation metrics and OpenAlex for metadata-rich indicators."

14.5 Maintenance Prompt

"When I update the CV, regenerate the site data, abstracts, and analytics without rebuilding the site manually."

14.6 GitHub Prompt

"Push the latest updates to GitHub and make sure GitHub Pages reflects the new version."

This prompt progression matters because it mirrors how the project matured. Codex was not asked to do everything perfectly in a single shot. It was asked to move the project forward one layer at a time.

---

15. Recommended Operating Pattern for Researchers

For researchers or project teams who want to reproduce this workflow, the most practical routine is:

1. keep the CV current
2. keep the project in a GitHub repository
3. use Codex for visible, incremental improvements
4. regenerate structured outputs whenever the CV changes
5. push only after local review
6. treat GitHub Pages as the public publication target

This model is especially useful once publications, grants, mentoring records, and service activities begin to grow faster than the researcher can comfortably manage by hand across multiple documents and websites.

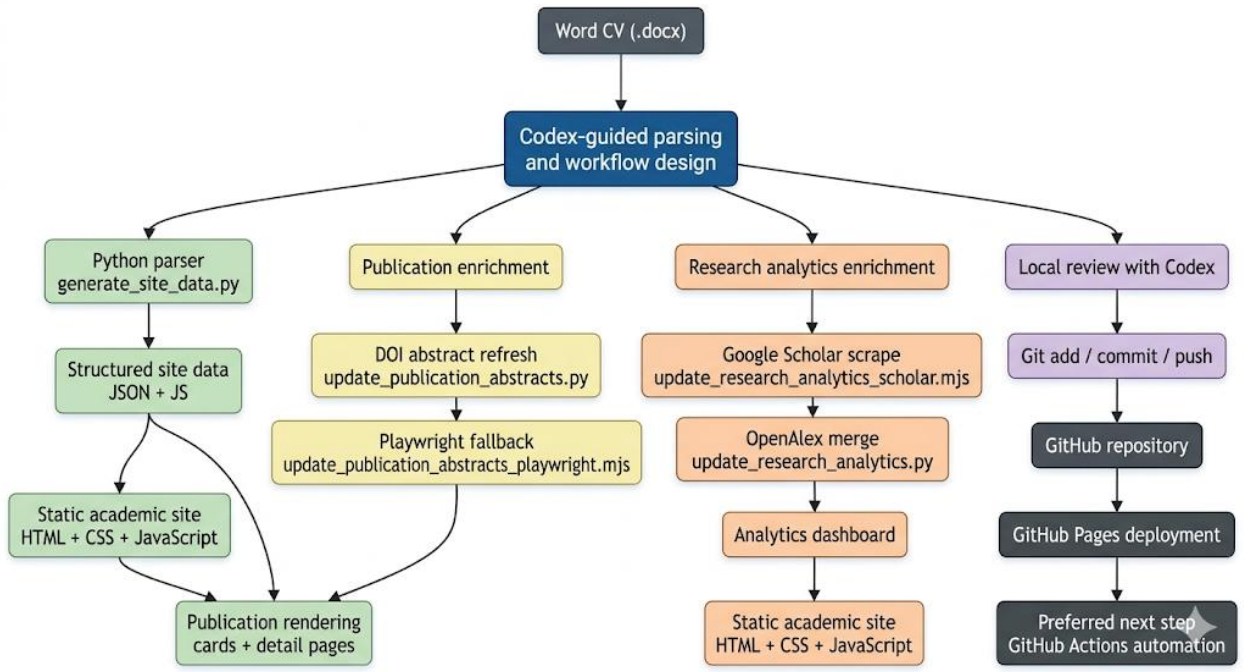
16. Strengths of the Codex + GitHub Model

The Codex + GitHub model is strong because it separates concerns clearly.

- Codex helps implement and refine.
- Python and Playwright transform and enrich.
- GitHub stores, versions, and publishes.

Together, they create a workflow that is reproducible, inspectable, and easier to maintain than a hand-edited academic website.

17. Workflow Diagram



18. Closing Note

The main takeaway from this workflow is that the important question is not only how to build a CV website, but how to work with Codex and GitHub together to maintain it as a living system. Codex helps turn requests into parsers, visual layouts, analytics, and refinements. GitHub turns those changes into a durable, shareable, publishable project. When those two layers are used together carefully, a CV can become the operational center of a real academic web presence rather than just a document stored on a desktop.

19. Codex Prompt Examples

The workflow becomes easier to reproduce when the researcher uses Codex with clear, concrete requests. The examples below are written as reusable prompts that can be adapted for another CV or another academic website project.

19.1 Initial Build Prompt

```
I am pasting my CV from Word below. Build a polished academic researcher website
from this CV content as a first working version. Then help me move toward a workflow
where the Word CV becomes the single source of truth. Use Python to parse the CV
into structured JSON, then render a static site with HTML, CSS, and JavaScript.
Include sections for overview, publications, grants, teaching, service,
affiliations, and honors. Make it look like a formal faculty/research profile rather
than a generic portfolio.
```

19.2 CV-to-Data Workflow Prompt

```
The first website version came from pasted CV content, but I now want the website to
regenerate from my actual Word CV rather than relying on manually edited data files.
Write a Python parser using python-docx that reads the CV and extracts appointments,
education, publications, grants, teaching, affiliations, service, and honors into
structured JSON and a browser-ready JS data file.
```

19.3 Publication Enrichment Prompt

```
Turn the publication section into a research archive rather than a plain list. Add
journal cover thumbnails, abstract previews, publication detail pages, pagination,
and corresponding-author labels where needed. Keep the main listing readable and
move the full abstract to the dedicated publication page.
```

19.4 Abstract Refresh Prompt

```
Create a publication abstract workflow that first uses DOI-based metadata collection
and then uses Playwright as a fallback for publisher pages that do not expose
abstracts cleanly. Save the result to a stable JSON file for the front end.
```

19.5 Analytics Dashboard Prompt

```
Add a researcher analytics dashboard. Use Google Scholar for total citations, h-
index, i10-index, yearly citation trend, and per-paper cited-by counts. Use OpenAlex
for venue distribution, topic clusters, coauthor network, and open-access
indicators. Use D3 for charts and the coauthor network, and clearly label the
dashboard as a month/year snapshot.
```

19.6 Automation Prompt

```
Create an orchestration script so that when I update the CV file, the project can
regenerate the structured site data, refresh publication abstracts, refresh Google
```

```
Scholar analytics, merge OpenAlex analytics, and leave the site ready for GitHub
Pages deployment.
```

19.7 GitHub Publishing Prompt

```
Review the changed files, stage the relevant updates, create a clean Git commit,
push the repository to GitHub, and confirm whether GitHub Pages has been updated
successfully.
```

19.8 Full End-to-End Prompt

```
I want you to build a complete CV-driven academic website workflow from my Word CV.
Assume I may begin by pasting CV content from Word into Codex, but the long-term
workflow should use the actual `.docx` as the source of truth. The site should
remain static, but the update process should be script-driven. Use Python for
parsing and orchestration, Playwright for fallback abstract extraction and Google
Scholar scraping, and GitHub Pages for deployment. Treat the CV as the single source
of truth and make the workflow reusable for future updates.
```

---

20. Refinement Prompts for Later Iterations

Once the base workflow is working, Codex is especially valuable for targeted refinement. The prompts below are intended for incremental improvement after the first usable version of the site already exists.

20.1 Visual Refinement

```
Keep the existing academic site structure, but refine the layout so it feels more
like a formal researcher homepage. Widen the content area, improve spacing, and
strengthen the visual hierarchy without turning it into a startup landing page.
```

20.2 Publication Presentation Refinement

```
Preserve the current publication data structure, but improve the publication cards
so the journal covers, abstract previews, and metadata are easier to scan. Keep the
publication section academically formal and visually organized.
```

20.3 Service and Affiliation Refinement

```
Redesign the service, editorial, and affiliation sections as thumbnail-supported
cards rather than plain text lists. Keep the content accurate, but make the sections
read more like an academic profile and less like a copied CV block.
```

20.4 Analytics Refinement

```
Keep the current analytics data sources, but refine the dashboard layout and labels
so users can immediately tell which metrics come from Google Scholar and which come
```

```
from OpenAlex. Make the dashboard read like a professional research profile
snapshot.
```

20.5 Automation Refinement

```
Preserve the current workflow, but make the update process more reusable and easier
to maintain. Clarify the orchestration script, document the update order, and keep
the project ready for future GitHub Actions automation.
```

20.6 GitHub Workflow Refinement

```
Review the current GitHub publishing workflow and identify any steps that still feel
manual or fragile. Recommend how to make the path from CV update to GitHub Pages
deployment cleaner and more reproducible without changing the static-site
architecture.
```

20.7 Documentation Refinement

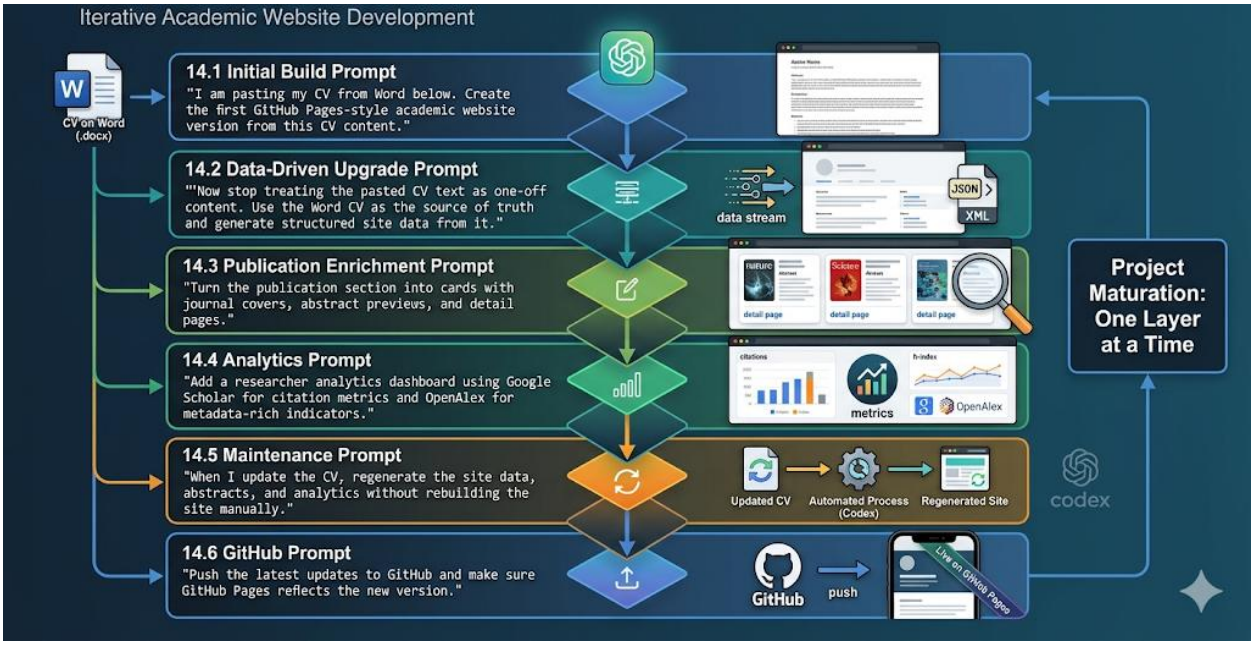
```
Rewrite the documentation so it explains the workflow more clearly for another
researcher who wants to reproduce it. Keep the document formal, practical, and
technically specific, with clear sections for Codex, Python scripts, Playwright,
GitHub, and GitHub Pages.
```

21. How to Use These Prompts

The most effective pattern is to use one prompt for each visible layer of progress. In practice, that means:

1. start with an initial build prompt
2. add data-driven parsing
3. enrich publications
4. add analytics
5. refine design and wording
6. push to GitHub
7. repeat when the CV changes

This prompt-based progression mirrors the real logic of the workflow. Codex works best when each request has a concrete target, a visible outcome, and a clear boundary.


Iterative Academic Website Development
CV on Word (.docx)
14.1 Initial Build Prompt
"I am pasting my CV from Word below. Create the first GitHub Pages-style academic website version from this CV content."
14.2 Data-Driven Upgrade Prompt
"'Now stop treating the pasted CV text as one-off content. Use the Word CV as the source of truth and generate structured site data from it."
data stream
JSON
XML
14.3 Publication Enrichment Prompt
"Turn the publication section into cards with journal covers, abstract previews, and detail pages."
detail page
detail page
detail page
14.4 Analytics Prompt
"Add a researcher analytics dashboard using Google Scholar for citation metrics and OpenAlex for metadata-rich indicators."
citations
metrics
h-index
OpenAlex
14.5 Maintenance Prompt
"When I update the CV, regenerate the site data, abstracts, and analytics without rebuilding the site manually."
Updated CV
Automated Process (Codex)
Regenerated Site
codex
14.6 GitHub Prompt
"Push the latest updates to GitHub and make sure GitHub Pages reflects the new version."
GitHub
push
Live on GitHub Pages
Project Maturation: One Layer at a Time